

MCQ-2

Topic: Conditional Statements, Python Loop, Python Function

Student Name: _____ **Date:** _____ **Score:** _____

Instructions: Select the correct option by filling or marking the checkbox ☐ next to it.

1. What is the output of the following Python code?

```
x = 2
if 0 < x < 3 == True:
    print("A")
elif 0 < x < 3 and 3 == True:
    print("B")
else:
    print("C")
```

- ☐ A
- ☐ B
- ☐ C
- ☐ Syntax Error

2. What is the exact output of this code snippet?

```
x = 0
while x < 5:
    x += 1
    if x == 3:
        continue
    print(x, end=" ")
```

- ☐ 1 2 4 5
- ☐ 0 1 2 4 5
- ☐ 1 2 followed by an infinite loop
- ☐ 1 2 3 4 5

3. What happens when you run this code?

```
i = 0
while i < 3:
    i += 1
    j = 0
    while j < 3:
        j += 1
        if j == 2:
            continue
        print(f"{i}{j}", end=" ")
```

- ☐ 11 13 21 23 31 33

- ☐ 11 21 31 22 31 33
- ☐ 11 12 21 22 31 33
- ☐ 11 13 21 23 32 33

4. What will be output?/

```
for i in range(1, 10, 3):
    print(i, end=" ")
```

- ☐ 1 4 7 10
- ☐ 1 4 7
- ☐ 3 6 9
- ☐ 1 2 3 4 5 6 7 8 9

5. What will be output?

```
items = [1, 2, 3]
for i in items:
    items.append(i)
    if len(items) > 5:
        break
print(items)
```

- ☐ [1, 2, 3, 1, 2, 3]
- ☐ [1, 2, 3, 1, 2]
- ☐ [1, 2, 3]
- ☐ This causes a syntax error.

6. What is the fundamental difference between a parameter and an argument in Python?

- ☐ An argument is used inside the function body, while a parameter is used in the main program.
- ☐ A parameter is the variable defined in the function definition, while an argument is the value passed to the function when it is called.
- ☐ They are the exact same thing
- ☐ A parameter is a value that cannot be changed, while an argument is a value that can be modified.

7. What will be the output of the following code?

```
x = 10
def my_func():
    x = 5
    print(x)
my_func()
print(x)
```

- ☐ 5,10
- ☐ 10,5
- ☐ 5,5

☐ 10,10

8. What is the primary purpose of using *args in a function definition?

- ☐ To enforce that only integers can be passed as arguments.
- ☐ To specify that the function does not accept any arguments.
- ☐ To pass a variable number of positional arguments to a function.
- ☐ To pass a fixed number of keyword arguments to a function.

9. What will be the output of the following code?

```
def display_items(*args):  
    print(type(args), args)  
  
display_items(10, "Python", True)
```

- ☐ <class 'tuple'> [10, 'Python', True]
- ☐ <class 'list'> [10, 'Python', True]
- ☐ <class 'tuple'> (10, 'Python', True)
- ☐ <class 'tuple'> ((10, 'Python', True))

10. If an asynchronous function encounters a blocking, synchronous operation (like "time.sleep(5)"), what happens to the overall execution?

- ☐ The event loop pauses the function and switches tasks automatically.
- ☐ The execution switches to another task to avoid blocks.
- ☐ The entire event loop stalls, freezing all other concurrent async tasks for 5 seconds.
- ☐ Python raises a BlockingIOError.

11. Consider the code below. What will be printed?

```
def test_func(x, *args, **kwargs):  
    print(x, args, kwargs)  
  
test_func(1, 2, 3, a=4, b=5)
```

- ☐ 1 (2, 3) {'a': 4, 'b': 5}
- ☐ 1 [2, 3] {'a': 4, 'b': 5}
- ☐ 1 (2, 3, 4, 5) {}
- ☐ SyntaxError

12. What is the main difference between Concurrency and Parallelism?

- ☐ Concurrency requires multiple physical CPU cores, while parallelism can run on a single-core processor using time-slicing.
- ☐ Concurrency is about dealing with multiple tasks overlapping in time by switching between them, whereas parallelism is about executing multiple tasks at the exact same physical instant.
- ☐ Parallelism is exclusively used for I/O-bound tasks like web scraping, while concurrency is used strictly for heavy mathematical calculations.

- ☐ There is no structural difference; the two terms are complete synonyms in modern software development.

13. What is the purpose of `asyncio.gather()` ?

- ☐ TO run coroutines sequentially
- ☐ To run multiple coroutines concurrently
- ☐ To cancel running task
- ☐ To create a new event loop

14. What will be the output of the following Python code?

```
def f(p, q):  
    return p%q  
  
print(f(0,2))  
print(f(2,0))
```

- ☐ 0
0
- ☐ Zero Division Error
Zero Division Error
- ☐ 0
Zero Division Error
- ☐ Zero Division Error
0

15. What will be the output of the following Python code?

```
def mk(x):  
    def mk1():  
        print("Decorated")  
        x()  
    return mk1  
  
def mk2():  
    print("Ordinary")  
  
p = mk(mk2)  
p()
```

- ☐ Decorated
Decorated
- ☐ Ordinary
Ordinary
- ☐ Ordinary
Decorated
- ☐ Decorated
Ordinary

16. What will be the output of the following Python code?

```

def f(x):
    def f1(*args, **kwargs):
        print("*"*5)
        x(*args, **kwargs)
        print("*"*5)
    return f1

@f
def p(m):
    print(m)
p("hello")

```

- ☐ *****
hello
- ☐ *****
hello

- ☐ *****
- ☐ hello

17. Which of the following must be defined in a recursive function?

- ☐ Base Case
- ☐ Iterative loop
- ☐ list comprehension
- ☐ conditional Statement

18. Which of the following represents a recursive data structure?

- ☐ Link List
- ☐ Tuple
- ☐ List
- ☐ Dictionary

19. Which of the following is a correct use of a lambda function with multiple arguments?

- ☐ lambda x,y : x+y
- ☐ lambda x y :x+y
- ☐ lambda xy: x+y
- ☐ lambda x,y = x+y

20. Which of the following statements is true about lambda functions?

- ☐ Lambda functions can have multiple expressions
- ☐ Lambda functions could have default argument
- ☐ Lambda functions can be named as regular function
- ☐ Lambda function return function